

Software Assignment Report
IMAGE COMPRESSION USING
TRUNCATED SVD

Avaneesh - EE25BTECH11030

November 7, 2025

Introduction

Singular Value Decomposition(SVD) is a decomposition of a rectangular matrix into three easily interpretable matrices.

SVD is one of the most powerful and versatile tools in applied mathematics, machine learning, and signal processing because it gives deep insight into a matrix's structure, stability, and dimensionality.

Summary of Strang's video

- SVD is the best decomposition as it can be done for any matrix.
- A rectangular matrix A is decomposed into three matrices U, V, Σ where U, V are orthogonal matrices and Σ is a diagonal matrix with its entries comprising singular values in descending order.

$$A = U\Sigma V^T$$

U is left singular matrix and V is right singular matrix.

- In case of symmetric positive definite matrix S , it can be decomposed as

$$S = Q\Lambda Q^T$$

where Q is orthogonal matrix and Λ is diagonal matrix containing Eigen values of the symmetric matrix.

- Let $A \in \mathbb{R}^{n \times m}$ where $n \geq m$ and $U \in \mathbb{R}^{n \times n}$, $V \in \mathbb{R}^{m \times m}$, $\Sigma \in \mathbb{R}^{n \times m}$

$$\begin{aligned} A^T A &= (U\Sigma V^T)(U\Sigma V^T) = V\Sigma^T U^T U\Sigma V^T \\ &= V\Sigma^T \Sigma V^T = V\Sigma^2 V^T \end{aligned}$$

$A^T A$ is sym and positive definite matrix, so V is matrix containing eigen vectors of $A^T A$ and Σ^2 the diagonal values are eigen values of $A^T A$. Similarly, U contains eigen vectors of matrix AA^T .

- We get

$$A \begin{pmatrix} V_1 & V_2 & \dots & V_r \end{pmatrix} = \begin{pmatrix} u_1 & u_2 & \dots & u_r \end{pmatrix} \begin{pmatrix} \sigma_1 & 0 & \dots \\ 0 & \sigma_2 & \dots \\ \dots & \dots & \dots \end{pmatrix}$$

where $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$

- product of eigen values = $\det(A)$
product of singular values = $\det(A)$
- The value of $u_1 \Sigma_1 v_1^\top$ is the most dominant and important data in the matrix.

Math

- Given a matrix A we need to calculate the matrix A_k which is the frobenius norm $\|A - A_k\|$ is minimum.
- For this I am using "Power Iteration with Deflation" method to solve for SVD to calculate the first k singular values ($\sigma_1, \sigma_2, \dots, \sigma_k$).

- We need to find

$$A_k = U_k \Sigma_k V_k^\top$$

- In this method I am first calculating the eigen vectors of $A^\top A$ which are the right singular vectors of A .

Let $B = A^\top A$, and the eigen values of B are $e_1, e_2, \dots$

- Now take a random vector $\mathbf{v}_1 = b_1 e_1 + b_2 e_2 + \dots$ where $b_1, b_2, \dots$ are some constants.

Consider a new vector

$$v_{new} = B v_1$$

$$v_{new} = b_1 B e_1 + b_2 B e_2 + \dots$$

as e are the eigen vectors of B , $B e_1 = \lambda_1 e_1$ where λ_1 is the eigen value of e_1 .

$$v_{new} = b_1 \lambda_1 e_1 + b_2 \lambda_2 e_2 + \dots$$

After finding v_{new} I am normalizing it.

- This step I am implementing repeatedly (about 20 - 30 times is sufficient) which makes the v_{new} with the eigen vector with the most dominant eigen value. In this way i was able to find the most important right singular vector.

Now $v_{new} = v_1$

For $\sigma_1 = \sqrt{\lambda_1}$ where $\lambda_1 = \|v_1\|$.

- For the remaining k-1 right singular vectors I will deflate the matrix B with the v_1 that I found in the before step.(i.e changing the matrix such that its singular value for v_1 will be 0.)

This matrix B is changed to $B - \lambda_1 v_1 v_1^\top$

- We can prove this by multiplying with vector $\mathbf{v}_1$ on both sides.

$$B\mathbf{v}_1 - \lambda_1 \mathbf{v}_1 \mathbf{v}_1^\top \mathbf{v}_1 = 0$$

Hence we removed the component of $\mathbf{v}_1$ from B .

- This same process is continued until the k right singular vectors and corresponding singular values are found and can form matrix V_k containing columns as the right singular vectors and Σ_k containing all diagonal entries as the singular values in descending order.
- We can find the vectors $u_1, u_2, \dots$ by

$$u_1 = \frac{A\mathbf{v}_1}{\sigma_1}, \dots$$

and hence form the matrix U_k containing the columns as $u_1, u_2, \dots$

- The output matrix = $U_k \Sigma_k V_k^\top$

Pseudo Code

Algorithm 1 Power Iteration: MAXLAMBDA(B, N)

Symmetric matrix $B \in \mathbb{R}^{n \times n}$

Dominant eigenvector v and corresponding eigenvalue λ_{max}

$v \leftarrow$ random vector of size n

$v \leftarrow v / \|v\|$ {Normalize the initial vector}

for $i = 1 \rightarrow 25$ **do** {number of iteration}

$v_{new} \leftarrow Bv$

$v \leftarrow v_{new} / \|v_{new}\|$ {Normalize the new vector}

end for

$\lambda_{max} \leftarrow v^\top (Bv)$ {This is Rayleigh Quotient}

return v, λ_{max}

Algorithm 2 Truncated SVD with Deflation: $\text{SVD}_{\text{TRUNCATED}}(A, \kappa)$

Matrix $A \in \mathbb{R}^{n \times m}$, rank k

Matrices $U_k \in \mathbb{R}^{n \times k}$, $S_k \in \mathbb{R}^k$, $V_k \in \mathbb{R}^{m \times k}$

Initialize U_k, S_k, V_k

$B \leftarrow A^\top A$

for $i = 1 \rightarrow k$ **do**

$(v, \lambda) \leftarrow \text{MAXLAMBDA}(B, m)$ {Find dominant component}

$\sigma \leftarrow \sqrt{\max(0, \lambda)}$ {so that there is no negative value under root}

$S_k[i] \leftarrow \sigma$ {Store singular value (as a vector)}

$V_k[i] \leftarrow v$ {Set i-th column of V_k as v }

$u \leftarrow Av$

$\sigma \leftarrow \max(\sigma, 10^{-12})$ {Avoid division by zero}

$u \leftarrow u/\sigma$

$U_k[i] \leftarrow u$ {Set i-th column of U_k as u }

$B \leftarrow B - \lambda vv^\top$ {Deflation of the matrix}

end for

return U_k, S_k, V_k

Algorithm 3 Image Reconstruction: $\text{RECONSTRUCTIMAGE}(U_k, S_k, V_k)$

$U_k \in \mathbb{R}^{n \times k}$, $S_k \in \mathbb{R}^k$, $V_k \in \mathbb{R}^{m \times k}$

Reconstructed matrix $A_k \in \mathbb{R}^{n \times m}$

$Sk_diag \leftarrow \text{diag}(S_k)$ {Create $k \times k$ diagonal matrix from S_k whose diagonal entries are entries of S_k }

$A_k \leftarrow U_k Sk_diag V_k^\top$ {Reconstructing the output matrix}

return A_k

Different Algorithms for computing SVD

There are four common methods for computing SVD:

1 Golub-Kahan-Reinsch (GKR)

Definition

It is the "gold standard" algorithm. It is used in making multiple libraries like LAPACK (which is used by NumPy, MATLAB, etc.) which uses SVD. It is a two-phase algorithm:

1. **Bidiagonalization:** Here, the matrix A is reduced to an upper bidiagonal matrix B using a series of Householder reflections.
2. **QR Iteration:** It is a variant of the QR algorithm which is iteratively applied to B to find its singular values, which are the same as A 's singular values.

This is the brief explanation on GKR.

Advantages

- **Numerically Stable**
- **Fast**
- **Guaranteed Convergence**

Disadvantages

- **Very Complex:** Very difficult to implement correctly from scratch.
- **Full SVD:** It computes the "full SVD", which is unnecessary as only the top k singular values are needed for compression.

2 Jacobi SVD

Definition

It is an iterative algorithm that applies a series of plane rotations (known as Jacobi rotations) to the matrix. The goal of this method is to apply these rotations in "sweeps" across the matrix to orthogonalize the columns (or rows). Eventually, it converges to a diagonal matrix whose diagonal entries are the singular values. This is a brief on Jacobi's approach.

Advantages

- **Simpler than GKR:** The core logic is a 2x2 rotation. This logic is much easier to understand and implement than GKR.
- **Accurate:** Can be very accurate, especially for small matrices.

Disadvantages

- **Slower:** Slower than GKR for larger matrices due to taking many sweeps for converging.
- **Full SVD:** Like GKR, it is a full SVD algorithm and is not useful in this case.

3 Power Iteration with Deflation

This is the method that I implemented which has been explained above.

Definition

This is a "truncated SVD" method that finds the singular components one by one. It relies on the fact that the eigenvectors of $A^T A$ are the right singular vectors (V) of A .

Advantages

- **Simple to Implement:** The power iteration and deflation steps are relatively simple to implement.
- **Truncated SVD:** Directly computes only the top k values, making it efficient if k is very small.

Disadvantages

- **Numerically Unstable:**
 - Computing $A^T A$ squares the condition numbers (which is a measure of how sensitive a function is to slight changes in inputs), which can lead to a significant loss of precision for small singular values.
 - The deflation step accumulates numerical errors so the value of v_k might have error.

4 Subspace Iteration (Orthogonal Iteration)

Definition

A more elegant way of "truncated SVD" method. It is essentially the power method applied to a block of k vectors simultaneously.

1. Start with a random $n \times k$ matrix V (representing k random vectors).
2. We have to use 2 steps iteratively : power and stabilization. Iterate:
 - $Z = (A^T A)V$ (The "power" step: applies B to all k vectors).
 - $[Q, R] = \text{qr}(Z)$ (The "stabilization" step: re-orthogonalizes the vectors via QR decomposition to prevent them all from collapsing to v_1).
 - $V = Q$
3. The columns of the final V matrix are the top k singular vectors.

Advantages

- **More Stable:** Vastly more numerically stable than power iteration with deflation because it avoids squaring condition number and subtraction for deflation. It uses stable QR decomposition instead.
- **Truncated SVD:** Efficiently finds the top k values at the same time.

Disadvantages

- **More Complex:** More difficult to implement than the deflation method, as it requires QR decomposition algorithm.
- **Cost of QR:** The QR decomposition step adds computational cost to each iteration.

Why I Used the Power Iteration with deflation method

Since I want to find the matrix corresponding with only the first k singular values, this method is very easy to implement and understand.

The other methods are very complex to implement from scratch.

The Subspace Iteration method is numerically more stable but difficult to implement the QR decomposition.

Compressed Images

4.1 Greyscale Images

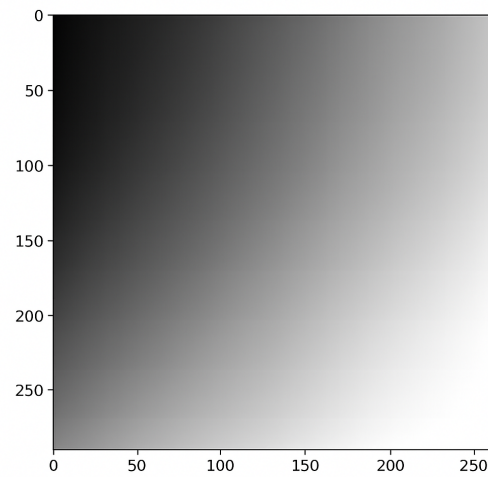


Figure 1: greyscale.png

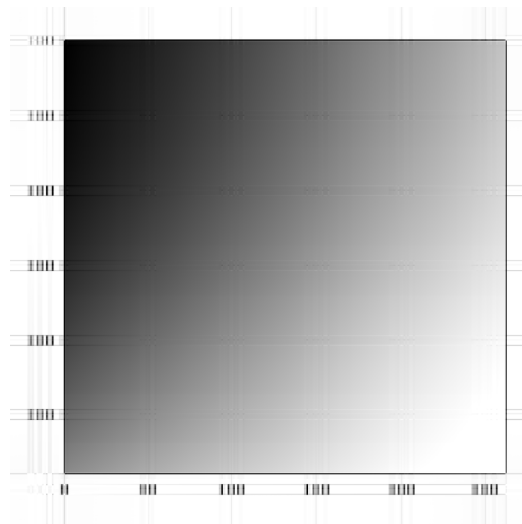


Figure 2: k=5

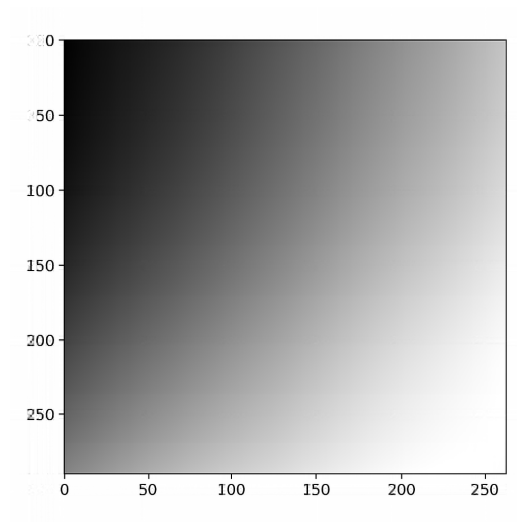


Figure 3: $k=20$

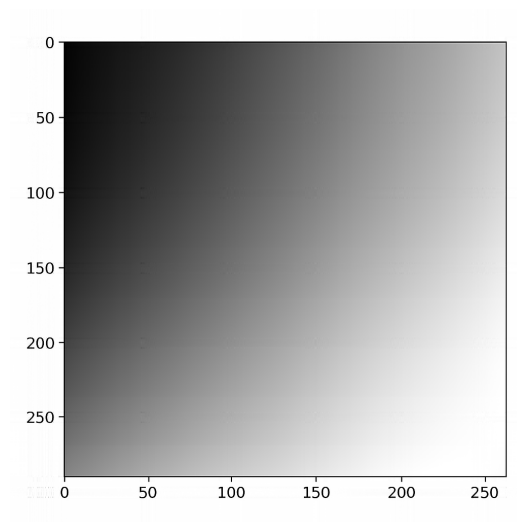


Figure 4: $k=50$

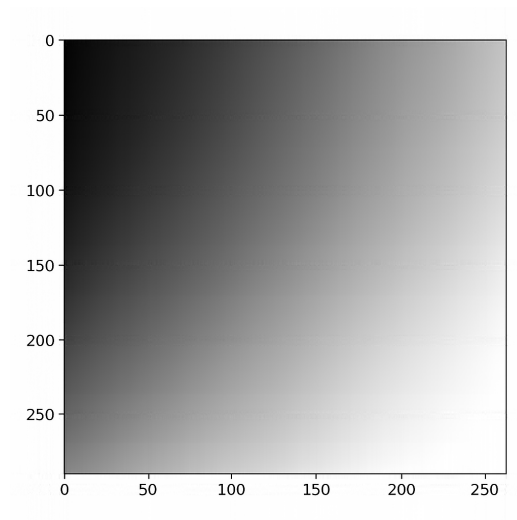


Figure 5: $k=100$

4.2 Globe Images



Figure 6: globe.jpg

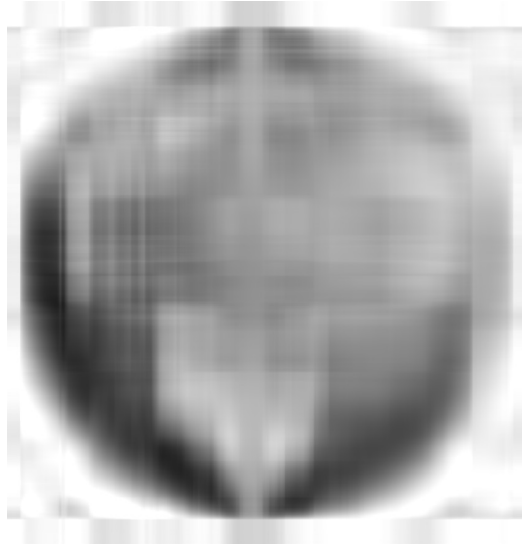


Figure 7: $k=5$



Figure 8: $k=20$



Figure 9: $k=50$



Figure 10: $k=100$

4.3 Einstein Images

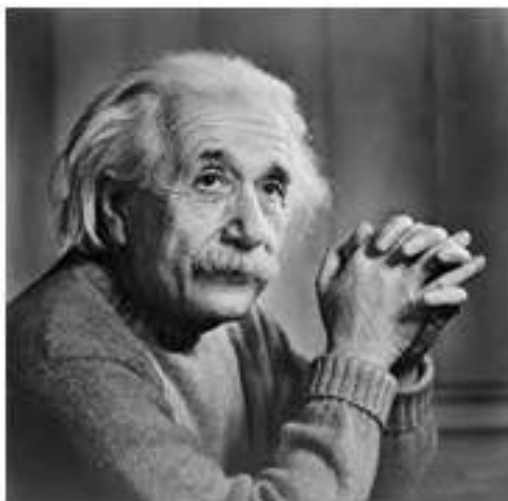


Figure 11: einstein.jpg



Figure 12: k=5



Figure 13: $k=20$

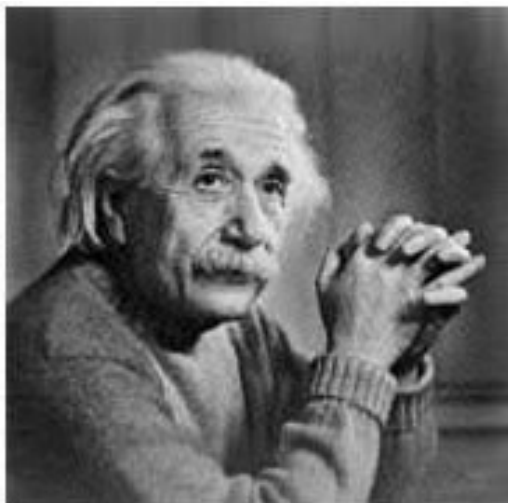


Figure 14: $k=50$

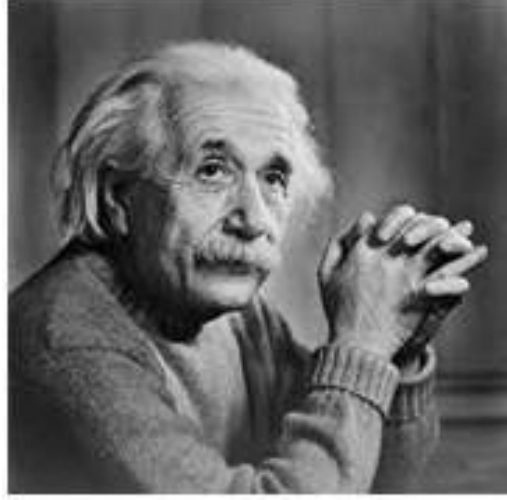


Figure 15: k=100

Error Analysis

The error rapidly decreases at first as $\|A - A_k\|_F$ denotes the energy of the elements of A which are not there in A_k . As the k values increase the amount of energy decreases in $A - A_k$.

Value of k	k=5	k=20	k=50	k=100
$\ A - A_k\ _F$	11146.3	3809.7	1504.1	981.2

Table 1: Frobenius Norms $\|A - A_k\|_F$ of greyscale.png

Value of k	k=5	k=20	k=50	k=100
$\ A - A_k\ _F$	20704.3	10636.6	6211.1	3719.2

Table 2: Frobenius Norms $\|A - A_k\|_F$ of globe.jpg

Value of k	k=5	k=20	k=50	k=100
$\ A - A_k\ _F$	4714.1	2127.1	885.0	199.2

Table 3: Frobenius Norms $\|A - A_k\|_F$ of einstein.jpg

Conclusion

SVD is a very useful and widely used decomposition which can be used for image compression. This is my approach on using SVD for image compression of greyscale images.